Aptos ULN 301

Security Assessment

May 27th, 2025 — Prepared by OtterSec

Bartłomiej Wierzbiński dark@osec.io

Robert Chen r@osec.io

Table of Contents

Executive Summary **2**

Overview 2

Key Findings 2

Scope 2

Findings **3**

General Findings **4**

OS-ULN-SUG-00 | Missing Validation Logic 5

OS-ULN-SUG-01 | Code Refactoring 6

OS-ULN-SUG-02 | Code Maturity 7

Appendices

Vulnerability Rating Scale **8**

Procedure **9**

© 2025 Otter Audits LLC. All Rights Reserved.

01 — Executive Summary

Overview

LayerZero Labs engaged OtterSec to assess the `aptos-uln-301` program. This assessment was conducted between May 5th and May 23rd, 2025. For more information on our auditing methodology, refer to Appendix B.

Key Findings

We produced 3 findings throughout this audit engagement.

In particular, we made recommendations for implementing proper validations and explicit checks to improve overall error handling (OS-ULN-SUG-00). We further suggested updating the codebase to improve the overall functionality (OS-ULN-SUG-01), and advised modifications for optimal efficiency and removal of unnecessary code, ensuring adherence to coding best practices (OS-ULN-SUG-02).

Scope

The source code was delivered to us in a Git repository at https://github.com/LayerZero-Labs/monorepo. This audit was performed against commit 3ac79c4. We further audited PR#2686, PR#2849, PR#2864 and PR#2881.

A brief description of the program is as follows:

Name	Description
aptos-uln-301	A compatibility module deployed on EndpointV1 that mirrors ULN302's behavior on EndpointV2.

© 2025 Otter Audits LLC. All Rights Reserved.

02 — Findings

Overall, we reported 3 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.

Severity	Count
CRITICAL	0
HIGH	0
MEDIUM	0
LOW	0
INFO	3

© 2025 Otter Audits LLC. All Rights Reserved.

03 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-ULN-SUG-00	There are several instances where proper validation is not performed, resulting in potential security issues.
OS-ULN-SUG-01	Recommendation for updating the codebase to improve the overall functionality.
OS-ULN-SUG-02	Suggestions to ensure adherence to coding best practices, enhance efficiency, and eliminate unutilized code instances.

© 2025 Otter Audits LLC. All Rights Reserved.

Missing Validation Logic

OS-ULN-SUG-00

Description

1. `msglib_v2_router::init` currently assumes `MsgLibConfig` is uninitialized, which may result in a runtime error if it is called again. Add a check to verify the existence of `MsgLibConfig` to improve error handling.

   >_ *msglib-v2/msglib-v2-0/msglib-v2-router/sources/msglib_v2_router.move* RUST

   ```rust
   /// The function is used to initialize the module after upgrade
   public entry fun init(account: &signer) {
       assert!(address_of(account) == @msglib_v2, EUNAUTHORIZED);
       move_to(account, MsgLibConfig {
           send_version: table::new()
       });
   }
   ```

2. `configuration::set_eid` currently allows setting `eid = 0`. Ensure to check that `eid > 0` to prevent invalid initializations and misconfigurations.

   >_ *msglib/msglib-v2/msglib-v2-0/uln-301/sources/internal/configuration.move* RUST

   ```rust
   public(friend) fun set_eid(eid: u32) {
       assert!(uln_301_store::eid() == 0, EALREADY_INITIALIZED);
       uln_301_store::set_eid(eid);
   }
   ```

3. `msglib_v2_router` utilizes a cached send-version from `MsgLibSendCapability` to route `get_ua_config` calls. This may result in incorrect routing if the user calls `endpoint::get_config` with a version that is not equivalent to the `send` version. Ensure to verify the cached version during config updates.

Remediation

Include the validations into the codebase.

© 2025 Otter Audits LLC. All Rights Reserved.

Code Refactoring OS-ULN-SUG-01

Description

1. `msglib_v2_router::set_ua_config` routes config updates based on the version stored in the `send` capability, but it does not update the cache. Enhancing it to also refresh the cached version will help to utilize the latest message library.
2. Some modules import from mock interfaces rather than actual production implementations. For example, `msglib_v2_router` imports `uln_301::router_calls` from a mock interface in `Move.toml`. Make sure the dependencies in `Move.toml` point to the correct modules.

Remediation

Incorporate the above refactors.

© 2025 Otter Audits LLC. All Rights Reserved.

Code Maturity OS-ULN-SUG-02

Description

1. `coin_to_fungible_asset` in `fa_converter` is an unnecessary wrapper around the public `coin::coin_to_fungible_asset`, adding no additional logic. It could be removed, and `coin::coin_to_fungible_asset` may be invoked directly without incurring overhead.

   >_ *msglib/msglib-utils/fa-converter/sources/fa_converter.move* RUST

   ```rust
   /// Convert a coin to a fungible asset
   public fun coin_to_fungible_asset<CoinType>(coin: Coin<CoinType>): FungibleAsset {
       coin::coin_to_fungible_asset(coin)
   }
   ```

2. Remove any un-utilized constants (such as `EVERSION_OUTDATED`) in `msglib_v2_router`.
3. `fa_converter::fungible_asset_to_coin` creates a new on-chain object and primary fungible store with each call, but deletes only the object, leaving behind an empty store. A more optimal approach will be to re-utilize a single primary fungible store. Additionally, the current converter may be unsuitable for dispatchable assets that charge deposit fees, as the withdrawal amount may exceed the remaining balance.

Remediation

Implement the above-mentioned suggestions.

© 2025 Otter Audits LLC. All Rights Reserved.

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

© 2025 Otter Audits LLC. All Rights Reserved.

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.

© 2025 Otter Audits LLC. All Rights Reserved.